SIMATS ENGINEERING

CSA3703 – SOFTWARE TESTING

ASSIGNMENT 2

Black-Box and White-Box Test Design for SwiftRide

Submission detail	Information
Student Name	Srikesh K
Register Number	192321040
Department / Section	Information Technology
Course Code and Name	CSA3703 – Software Testing
Assignment Nature	Individual
Duration	1 week
Total Marks	50
Date of Submission	4-11-26
Faculty Name	Dr Gnanajeyaraman Rajaram

SRIKESH K - 192321040

Executive Summary

This report designs and compares black-box and white-box test techniques for SwiftRide, a cab-booking and fare-calculation application. The work follows the assignment specification: fare slabs, waiting charges, night surcharge, RIDE50 promotion, ₹800 cap, wallet eligibility, and the ride life cycle. The report compares alternative methods, gives representative test data, explains coverage measures, and proposes a practical hybrid suite for a two-day testing window.

Important execution note: code examples and test tables below are reproducible design artefacts. Values described as predicted or illustrative must be replaced with actual output from the student's local pytest/coverage.py/mutmut run before final submission. The assignment explicitly requires results from your own execution; do not present an unexecuted estimate as a measured result.

Contents

- 1. Scenario, assumptions and reference implementation
- 2. Task 1 – Test design strategies
- 3. Task 2 – Random testing
- 4. Task 3 – Equivalence class partitioning
- 5. Task 4 – Boundary value analysis
- 6. Task 5 – Decision table, state transition and pairwise testing
- 7. Task 6 – Test adequacy criteria
- 8. Task 7 – Decision, condition/decision and MC/DC coverage
- 9. Task 8 – Control-flow graphs and cyclomatic complexity
- 10. Task 9 – Integrated test suite
- 11. Task 10 – Adapting to changed requirements
- 12. Overall recommendation and references

1. Scenario, Assumptions and Reference Implementation

SwiftRide accepts ride distance, waiting time, booking hour, promo code, wallet balance and payment mode. Distances are recorded to 0.1 km. The test design uses the rules supplied in the assignment and avoids silently inventing business behaviour.

Rule	Interpretation used in this report
Distance	Valid range is 0.1–60.0 km inclusive, precision 0.1 km.
Fare slab 1	For km ≤ 2, fare = ₹50.
Fare slab 2	For 2 < km ≤ 10, fare = ₹50 + (km − 2) × ₹12.
Fare slab 3	For km > 10, fare = ₹146 + (km − 10) × ₹10.
Waiting	First 5 minutes free; each minute above 5 adds ₹2. Valid wait is 0–30 minutes.
Night surcharge	Multiply by 1.25 when hour ≥ 23 or hour < 5; hours 23, 0, 1, 2, 3, 4 are night.

Promo	RIDE50 deducts ₹50 if fare after waiting and night surcharge is at least ₹200; promo usage once per passenger.
Fare cap	After promo, fare is capped at ₹800. This ordering follows the supplied listing.
Wallet	Wallet is allowed if balance ≥ final fare; otherwise return WALLET_LOW.
Lifecycle	Requested → Accepted → Arrived → Started → Completed. Cancellation allowed before Started; ₹30 fee after Arrived.

Assumptions requiring confirmation: (1) invalid wait, hour, mode and wallet values are not explicitly handled in Listing 1; tests should document whether they are rejected by input validation or by the function. (2) Fare calculation rounds to two decimals only at return. (3) The promo threshold is checked before deduction and after waiting/night multiplication, exactly as the listing states. (4) The supplied code does not implement promo-use history or ride lifecycle; those require separate modules.

Reference implementation – Listing 1

```
def calculate_fare(km, wait, hour, promo, wallet, mode):
    if km <= 0 or km > 60:
        return "INVALID"
    if km <= 2:
        fare = 50
    elif km <= 10:
        fare = 50 + (km - 2) * 12
    else:
        fare = 146 + (km - 10) * 10
    if wait > 5:
        fare = fare + (wait - 5) * 2
    if hour >= 23 or hour < 5:
        fare = fare * 1.25
    if promo == "RIDE50" and fare >= 200:
        fare = fare - 50
    if fare > 800:
        fare = 800
    if mode == "WALLET" and wallet < fare:
        return "WALLET_LOW"
    return round(fare, 2)
```

Reference implementation – Listing 2

```
def find_driver(ids, target):
    low = 0
    high = len(ids) - 1
    while low <= high:
        mid = (low + high) // 2
        if ids[mid] == target:
            return mid
        elif ids[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

For find_driver, the precondition is that ids is sorted in ascending order. Tests must include a first, middle and last element, a missing target below/between/above the list, an empty list and a one-element list.

2. Task 1 – Test Design Strategies (4 Marks)

Specification-based testing derives expected results from business rules without inspecting code. Structure-based testing uses statements, branches and conditions in Listing 1. Fault-based testing targets likely mistakes such as changed operators, constants and omitted checks.

Strategy	Strength	Likely blind spot	Effort under time limit
Specification-based	Finds requirement violations and missing/wrong business behaviour; supports traceability.	May miss internal branches or coding faults not reflected in the specification.	Medium; rules must be converted into independent expected results.
Structure-based	Reaches true/false outcomes and exposes unexecuted logic.	Cannot prove that the code implements the correct requirement; missing functionality has no branch to cover.	Medium; requires code understanding and coverage tooling.
Fault-based	Targets realistic defects and checks whether tests distinguish faulty versions.	Only detects faults anticipated by the tester/mutant generator.	Higher initially; valuable for checking suite strength.

Example six seeded faults and discriminating tests

Fault ID	Seeded change	Discriminating input	Expected detection
F1	km <= 2 changed to km < 2	km=2.0, day, no wait	Fare should remain ₹50; faulty branch enters second slab.
F2	hour >= 23 changed to hour > 23	km=3, hour=23	Night surcharge must apply at 23:00.
F3	promo threshold 200 changed to 150	fare before promo = ₹180, promo=RIDE50	Promo must not apply.
F4	wait > 5 changed to wait >= 5	wait=5	No waiting charge should be added.
F5	cap 800 changed to 900	long ride, no promo	Final fare must not exceed ₹800.
F6	wallet < fare changed to wallet <= fare	wallet exactly equals fare	Payment must be accepted.

Comparison metrics should be computed from actual runs: faults found = number of seeded faults killed; faults per test = faults found ÷ test count; design effort = measured minutes spent creating and reviewing that suite. Record results rather than inventing measurements.

Strategy	12-test suite design emphasis	Faults found (fill after run)	Faults/test	Effort (min)
Specification-based	One test per fare rule, promotion, cap, wallet rule and invalid input.	____	____	____
Structure-based	True/false for each decision, plus condition combinations.	____	____	____
Fault-based	At least one input specifically killing each seeded fault.	____	____	____

Recommendation: apply specification-based tests first to anchor the oracle, add boundary and branch tests to expose edge logic, then use mutation/fault-based testing to assess whether the suite is sensitive to realistic defects. If requirements are incomplete, all three become weaker: specification-based design lacks an oracle, structure-based design can only test what exists, and fault-based design depends on the completeness of its fault catalogue.

LLM critique: a test set generated only from source code usually resembles structure-based testing because it can see operators and branches. It may miss incorrect business rules if the prompt omits the requirement. Its output must be checked for independent expected values, exact boundaries and wallet equality.

3. Task 2 – Random Testing Approaches (4 Marks)

Approach	Generation rule	Best use	Main limitation
Uniform random	Sample km, wait, hour, promo, wallet and mode uniformly from their allowed domains.	Broad exploratory checks and unexpected combinations.	Rare boundaries and realistic combinations may be under-sampled.
Operational profile	70% distances 2–10 km; 20% >10 km; 10% ≤2 km; 15% night; 5% promo.	Reliability estimate for production-like usage if the profile is accurate.	Biased if usage assumptions are stale; may under-sample rare but severe cases.
Adaptive random	Choose new inputs maximally distant from previously selected inputs.	Spreads coverage across numeric/parameter space.	Distance metric and categorical encoding affect what "far apart" means.

Boundary-hit probability

Assume km is continuous-uniform on 0.1–60.0 km, with range width 59.9 km. A band within 0.1 km on one side of a boundary has approximate probability 0.1/59.9 = 0.00167 = 0.167%. If "within 0.1 km" means ±0.1 km around the boundary, the band width is 0.2 km and probability ≈ 0.2/59.9 = 0.00334 = 0.334% per interior boundary, subject to endpoint clipping. Thus 40 tests have only about 1−(1−0.00334)^40 ≈ 12.5% chance to hit the ±0.1 km band around one boundary. Exact boundary values are even less likely under continuous random sampling.

Reproducible random generation example

```
import random
def uniform_case(seed):
    r = random.Random(seed)
    return (round(r.uniform(0.1, 60.0), 1),
            r.randint(0, 30), r.randint(0, 23),
            r.choice(["", "RIDE50"]),
            round(r.uniform(0, 900), 2),
            r.choice(["WALLET", "UPI", "CASH"]))
```

For each approach, generate 40 tests and repeat with 10 fixed seeds. Record mean faults found, range or standard deviation, and average test index at which the first fault is detected. A trustworthy oracle can be an independently written reference model or hand-verified expected values for a smaller golden suite. Do not use the same faulty implementation as both system-under-test and oracle.

Approach	Runs	Mean faults found	Spread (SD or range)	Mean tests to first fault
Uniform random	10 × 40	____	____	____
Operational profile	10 × 40	____	____	____
Adaptive random	10 × 40	____	____	____

Random testing is useful for broad exploration and regression, but designed tests must cover fare boundaries, hour 23, wait 5, promo threshold ₹200, wallet equality and state transitions. Operational-profile random testing supports

a reliability estimate only when its distribution matches real usage and the oracle is correct; uniform random testing is not an unbiased estimate of production reliability when production inputs are non-uniform.

4. Task 3 – Equivalence Class Partitioning (5 Marks)

Equivalence class partitioning (ECP) groups inputs expected to behave similarly and selects representative values. Robust partitioning adds invalid classes and tests each invalid class separately so one invalid input does not mask another. Classification-tree partitioning combines parameter classes systematically.

Parameter	Valid classes (representatives)	Invalid classes (representatives)
km	K1: 0.1–2.0 (1.0); K2: >2–10 (5.0); K3: >10–60 (20.0)	K0: ≤0 (0.0); K4: >60 (60.1)
wait	W1: 0–5 (5); W2: 6–30 (6)	W0: <0 (-1); W3: >30 (31)
hour	H1: 5–22 (12); H2: 23–23 (23); H3: 0–4 (2)	H0: <0 (-1); H4: >23 (24)
promo	P1: absent/other (empty); P2: RIDE50	P0: malformed/null if API rejects it
wallet	B1: below fare (fare−0.01); B2: equal (fare); B3: above (fare+0.01)	B0: negative balance
mode	M1: WALLET; M2: UPI; M3: CASH	M0: unsupported mode

Representative test design

ID	Input summary	Classes covered	Expected result
ECP-01	km=1, wait=0, hour=12, no promo, CASH	K1,W1,H1,P1,M3	₹50.00
ECP-02	km=5, wait=6, hour=12, no promo, CASH	K2,W2,H1,P1,M3	₹88.00
ECP-03	km=20, wait=0, hour=12, no promo, CASH	K3,W1,H1,P1,M3	₹246.00
ECP-04	km=20, hour=23, RIDE50, CASH	K3,H2,P2	₹257.50 (246×1.25−50)
ECP-05	km=5, wallet=0, WALLET	K2,B1,M1	WALLET_LOW
ECP-06	km=5, wallet=88, WALLET; wait=6	K2,W2,B2,M1	₹88.00 accepted
ROB-01	km=0.0; other fields valid	K0 only invalid	INVALID
ROB-02	wait=31; other fields valid	W3 only invalid	Validation error expected from input layer; Listing 1 does not define it.
ROB-03	hour=24; other fields valid	H4 only invalid	Validation error expected from input layer; Listing 1 does not define it.
ROB-04	mode=CRYPTO; other fields valid	M0 only invalid	API should reject unsupported mode; function currently falls through to fare.

Variant comparison: valid-only ECP uses the fewest tests but misses invalid-input handling; robust partitioning produces more tests and gives clearer diagnosis; a classification tree is best for selecting combinations across fare, time, promo and payment classes. The recommended approach is robust partitioning for each parameter plus a small classification-tree combination suite for interactions.

The “all values in a class behave alike” assumption can break when independent rules interact. For example, a 10 km ride at night with RIDE50 can cross the promo threshold only after applying the night multiplier. Split a coarse class whenever a rule boundary, interaction, or different expected action exists. LLM-generated partitions should be reviewed for missing negative values, >60 km, unsupported mode, wallet-balance relations and the exact 10 km boundary.

5. Task 4 – Boundary Value Analysis (5 Marks)

Parameter	Boundary	Values to test (precision-aware)
km	Minimum 0.1	0.0 invalid, 0.1, 0.2
km	Slab boundary 2.0	1.9, 2.0, 2.1
km	Slab boundary 10.0	9.9, 10.0, 10.1
km	Maximum 60.0	59.9, 60.0, 60.1 invalid
wait	Free waiting boundary 5	4, 5, 6
wait	Maximum 30	29, 30, 31 invalid
hour	Night start/end	4, 5 and 22, 23

Methods and test counts

Method	Selection method	Estimated suite size for km/wait/hour	Trade-off
Two-value boundary	Boundary and nearest value on the other side for each threshold.	Per listed boundary pair: compact, about 14–18 tests depending on shared values.	Efficient for off-by-one errors; may miss interactions and minimum/maximum invalid values.
Robust boundary	For each parameter: min−, min, min+, nominal, max−, max, max+.	7 values × 3 parameters = 21 tests if one parameter varies at a time.	Catches invalid extremes; manageable and diagnostic.
Worst-case boundary	Five valid values per parameter combined in all combinations.	$5^3 = 125$ tests for km, wait and hour.	Strong combination coverage but consumes more of the test budget.

Use a nominal distance of 5.0 km, nominal wait 10 minutes and nominal hour 12 for one-factor-at-a-time robust tests. For worst-case testing, use five valid values per parameter (minimum, minimum+, nominal, maximum−, maximum) as specified; invalid values are covered separately.

Input	Expected calculation	Why it matters
km=2.0, day	₹50.00	Exactly 2 km stays in flat slab.
km=2.1, day	₹51.20	₹50 + 0.1×₹12.
km=10.0, day	₹146.00	₹50 + 8×₹12.
km=10.1, day	₹147.00	₹146 + 0.1×₹10.
km=3, wait=5, hour=23	₹62.50	No waiting charge; night multiplier applies.

Floating-point arithmetic can represent decimal values such as 10.1 approximately in binary. The test oracle should use Decimal or round the final expected fare to two decimal places, matching the specification. Boundary testing would likely catch the wrong slab and missing 11 pm surcharge if tests explicitly include 10.0/10.1 km and hour 23; it does not replace promo, wallet, interaction or lifecycle tests.

Recommendation: robust boundary testing offers the best balance for the two-day window. Use worst-case combinations only for high-risk combinations or where pairwise results show interaction risk.

6. Task 5 – Decision Table, State Transition and Pairwise Testing (5 Marks)

6.1 Decision table

Rule	Fare ≥ ₹200?	Promo=RIDE50?	Wallet balance ≥ final fare?	Mode	Action
R1	No	Yes	N/A	Any	No promo; return calculated fare or WALLET_LOW if wallet short.
R2	Yes	No	N/A	Any	No discount; continue cap/payment checks.
R3	Yes	Yes	N/A	Any	Subtract ₹50, then apply cap and payment rule.
R4	Any	Any	No	WALLET	Return WALLET_LOW.
R5	Any	Any	Yes	WALLET	Accept wallet payment.
R6	Any	Any	N/A	UPI/CASH	Continue with final fare; wallet balance irrelevant.

Decision tables are ideal for combinations of promo threshold, discount, cap and payment. Conditions should be made mutually understandable; for example, the fare ≥ ₹200 condition is evaluated after waiting and night charges but before promo deduction.

6.2 Ride state-transition model

Current state	Event	Next state	Expected result
Requested	Driver accepts	Accepted	Booking confirmed.
Accepted	Driver arrives	Arrived	Arrival recorded.
Arrived	Ride starts	Started	Trip timer begins.
Started	Ride ends	Completed	Final fare is calculated.
Requested	Passenger cancels	Cancelled	No cancellation fee.
Accepted	Passenger cancels	Cancelled	No cancellation fee under supplied rule.
Arrived	Passenger cancels	Cancelled	₹30 cancellation fee.
Started	Passenger cancels	Rejected / no state change	Cancellation prohibited.
Requested	Complete event	Rejected / no state change	Cannot complete before acceptance/arrival/start.
Completed	Start event	Rejected / no state change	Terminal state cannot restart.

State flow: Requested → Accepted → Arrived → Started → Completed. Cancellation branches from Requested, Accepted and Arrived; the Arrived branch applies ₹30. Attempts to cancel from Started or Completed must be rejected.

6.3 Pairwise testing

Pairwise testing chooses a compact set in which every pair of parameter values appears at least once. It is useful for km class, wait class, hour class, promo, wallet class and mode, but does not guarantee all six-way combinations or all business rules. The full Cartesian product grows quickly: if the class counts are 3×2×3×2×3×3, there are 324 combinations. A pairwise generator should produce a much smaller covering array; record the actual generated size and pair coverage from the selected tool.

```
# Example test dimensions for a pairwise generator
parameters = {
  "km_class": ["short", "middle", "long"],
```

```
  "wait_class": ["free", "charged"],
  "hour_class": ["day", "night", "boundary_23"],
  "promo": ["none", "RIDE50"],
  "wallet": ["low", "equal", "high"],
  "mode": ["WALLET", "UPI", "CASH"],
}
```

Technique	Faults it is best positioned to find	Limitation
Decision table	Wrong promo threshold, cap order, wallet eligibility combinations.	Does not naturally cover event sequences.
State transition	Cancellation after start, impossible completion/start sequences.	Does not exhaust fare input combinations.
Pairwise	Interactions between any two parameter classes with fewer cases than full product.	Can miss three-way or narrow boundary interactions.

Use all three rather than selecting one universally: decision tables for fare/payment rules, state-transition testing for ride lifecycle, and pairwise testing for broad combination coverage. Review LLM-generated decision tables against every stated rule and reject impossible transitions such as Completed → Started.

7. Task 6 – Test Adequacy Criteria (4 Marks)

Criterion	Meaning	Strength	What it cannot guarantee
Statement coverage	Percentage of executable statements run.	Finds dead/unexecuted statements and basic omissions in test execution.	A statement may run without its faulty outcome being checked.
Branch coverage	Percentage of decision outcomes (true/false) exercised.	Forces both outcomes of if/elif/while decisions.	May not show each simple condition independently changes a decision.
Mutation score	Killed non-equivalent mutants ÷ total non-equivalent mutants × 100%.	Checks whether tests detect realistic code changes.	Mutant set quality matters; equivalent mutants must be excluded.

Measure statement and branch coverage with coverage.py after running the suite. Generate at least 20 mutants, such as changing <= to <, 12 to 11, 1.25 to 1.2, 200 to 150, 800 to 900, and < to <=. Mutation score = killed mutants / non-equivalent mutants × 100%. Record surviving mutants and determine whether they represent weak tests, unreachable conditions or equivalent behaviour.

Measure	Observed result from local run	Uncovered / surviving items
Statement coverage	____ %	_____
Branch coverage	____ %	_____
Mutation score	____ %	_____
Runtime for full suite	____ seconds	Must meet under 2 seconds per automated test.

A suite can achieve 100% statement coverage while using only one outcome of a condition or failing to assert the correct fare. For example, a test may execute the promo deduction line with a wrong threshold but never test fares between ₹150 and ₹199. Therefore, 100% statement coverage is not proof of correctness.

Recommended release gate: at least 90% branch coverage as the assignment target, no surviving high-risk mutants related to slab boundaries, night surcharge, promo, cap or wallet equality, and all critical requirements covered by explicit assertions. Coverage is evidence of execution, not a substitute for requirements-based testing.

8. Task 7 – Decision, Condition/Decision and MC/DC (5 Marks)

Decision	Simple conditions	Test focus
Line 2: km <= 0 OR km > 60	A: km <= 0; B: km > 60	A true, B false; A false, B true; both false for valid km.
Line 12: hour >= 23 OR hour < 5	A: hour >= 23; B: hour < 5	hour=23 isolates A; hour=4 isolates B; hour=12 makes both false.
Line 14: promo == RIDE50 AND fare >= 200	A: promo matches; B: fare ≥ ₹200	RIDE50 with fare 200 vs 199; non-promo with fare ≥200.
Line 18: mode == WALLET AND wallet < fare	A: mode is WALLET; B: wallet < fare	Wallet low balance; wallet equal; non-wallet mode with low wallet.

Criterion	Requirement	Example for line 14	Example for line 18
Decision coverage	Each decision true and false.	T: RIDE50, fare=220; F: no promo, fare=220.	T: WALLET and balance < fare; F: UPI.
Condition/decision	Each simple condition true/false and each decision true/false.	Need promo match true/false and threshold true/false, plus both decision outcomes.	Need wallet mode true/false and balance comparison true/false.
MC/DC	Each condition independently changes the decision while other conditions are held fixed.	Pair: RIDE50/fare=220 vs no-promo/fare=220; pair: RIDE50/fare=220 vs RIDE50/fare=199.	Pair: WALLET/low vs UPI/low; pair: WALLET/low vs WALLET/equal.

For line 12, MC/DC pairs should demonstrate that each sub-condition independently changes the OR result: (hour=23 vs hour=22) isolates hour ≥23 while hour <5 remains false; (hour=4 vs hour=5) isolates hour <5 while hour ≥23 remains false. A decision-only suite that tests hour=23 and hour=12 could still miss a defect changing hour <5 to hour <4.

Short-circuit evaluation means the second operand of an OR may not be evaluated when the first is true, and the second operand of an AND may not be evaluated when the first is false. Coverage tools may report condition evaluation differently; ensure test data makes each simple condition observable. MC/DC is most valuable for the fare function's compound rules; branch coverage is a practical baseline for less critical utility code.

9. Task 8 – Control-Flow Graphs and Cyclomatic Complexity (4 Marks)

For Listing 2, the control-flow decisions are the while condition, equality comparison and less-than comparison. There are three predicate nodes, so V(G) = predicates + 1 = 3 + 1 = 4.

Graph granularity	Nodes and edges concept	Expected V(G)	Use
Statement-level	Separate executable statements and branch outcomes.	4	Most detail; good for teaching and line-level traceability.
Basic-block	Merge straight-line statements; keep decisions as block ends.	4	Readable balance of precision and simplicity.
Decision-to-decision path	Represent entry/exit and decision nodes, connect complete paths between them.	4	Compact overview of independent paths.

Independent paths with sample inputs

Path	Route through logic	Input example	Outcome

P1	Loop not entered → return -1	ids=[], target=5	-1
P2	Loop → equality true → return index	ids=[2,5,8], target=5	index 1
P3	Loop → equality false → ids[mid] < target → move low right → find	ids=[2,5,8], target=8	index 2
P4	Loop → equality false → ids[mid] > target → move high left → no match	ids=[2,5,8], target=4	-1

Cyclomatic complexity is invariant under reasonable graph granularity when the same control structure is represented correctly. Use V(G)=E−N+2P for a single connected component (P=1), hence E−N+2; cross-check with predicate count + 1 and regions. A correct graph must include both outcomes of each decision and the back edge from the loop body to the while condition.

The sorted-list precondition matters: binary search behaviour is not guaranteed for unsorted input. Test first/middle/last hits, empty/singleton lists, missing targets and duplicate IDs if duplicates are permitted. Basic-block graphs are recommended for the team because they remain readable while preserving complexity and independent-path reasoning.

10. Task 9 – Integrated Test Suite (6 Marks)

Suite	Composition	Expected advantage	Risk
A. Basis-path	Independent paths based on CFG, one or more tests per basis path.	Explicit control-flow path coverage.	May not cover important business boundaries or interactions.
B. Black-box + pairwise	ECP, BVA, decision table, state transitions and pairwise.	Strong requirement and interaction coverage.	May leave source branches/conditions uncovered.
C. Hybrid	Start with black-box tests; add tests for uncovered branches, conditions and surviving mutants.	Combines requirement traceability with implementation evidence.	Needs disciplined coverage/mutation review.

Integrated workflow

Step	Activity	Output / gate
1	Read rules and list assumptions	Requirement-to-test traceability matrix.
2	Design black-box cases	ECP, BVA, decision-table and state-transition cases.
3	Run tests and measure coverage	pytest result, runtime, coverage report.
4	Add white-box top-up	Tests for uncovered branches/conditions.
5	Run mutation testing	Killed/surviving mutant list; investigate critical survivors.
6	Create regression pack	Prioritized tests, test data, expected values and reproducible command.

Design decision table

Decision	Justification (evidence to record)	What if not?
Use hybrid suite	Requirements coverage plus ≥90% branch coverage and high-risk mutants killed.	A pure basis-path suite can miss incorrect fare rules; a pure black-box suite may miss branches.
Prioritize fare boundaries	Reported defects cluster at 2 km, 10 km, 5 min and 23:00.	Production complaints may recur even if broad coverage looks high.
Keep lifecycle tests separate	Listing 1 does not implement ride state transitions.	Do not claim fare-function tests prove lifecycle behaviour.

Keep each test <2 seconds	Measure individual test duration in CI.	Slow tests reduce feedback and risk violating the constraint.

Basis paths of Listing 1 should be derived from its control-flow graph, but some syntactic paths are infeasible because the business predicates constrain values. For example, a fare of ₹50 for a ride ≤2 km cannot reach the ₹200 promo threshold under ordinary day conditions; the ₹800 cap cannot be reached for rides ≤10 km. The exact infeasibility argument should include maximum waiting and night multiplier where relevant.

For the train-ticket concession program, paths depend on age/student/senior status and ticket type; for date validation, paths depend on month length, leap year and day bounds. Each needs its own CFG and V(G); do not reuse SwiftRide's complexity number.

Model-based or LLM-assisted test generation can accelerate draft generation, but generated suites need human verification, independent expected values, requirement traceability and local execution. Recommended final suite: C, the hybrid suite.

11. Task 10 – Adapting to Changed Requirements (8 Marks)

New requirements add pooled rides, surge multiplier 1.0–3.0 in 0.1 steps, toll/airport charges, 5% GST, loyalty discounts and time-window promos. The fare function grows to about 35 decision points, releases are weekly, and regression must complete in 10 minutes.

Changed assumption	Why Task 9 assumption fails	Risk if old suite is unchanged
Small decision structure	About 35 decision points increase branch and path complexity.	Old suite misses new decision outcomes and interactions.
Input independence	Surge, zone, demand, pooled riders, toll, airport and loyalty may interact.	Pairwise only may miss high-order pricing interactions.
Stable specification	Promos are time-window based and releases are weekly.	Hard-coded test data becomes stale and tests become brittle.
Runtime flexibility	Regression now has a 10-minute total limit.	Full Cartesian product or exhaustive paths may exceed time.

Compare adaptation strategies

Strategy	How to apply	Advantages	Limitations
A. Retain + prioritise	Keep old suite; run high-risk and recently changed tests first.	Fastest initial implementation; preserves known defect tests.	Cannot validate rules absent from old suite.
B. Modify decision tables + orthogonal arrays	Add new pricing conditions, retain boundaries, use pairwise/orthogonal arrays.	Systematic interaction coverage with controlled test count.	Needs maintenance as new factors/rules are added.
C. Redesign model-based + metamorphic	Build a fare model and generate cases; check monotonicity and invariants.	Scales better to complex combinations and frequent releases.	Model must be independently reviewed; properties need exceptions for discounts/caps.

Example metamorphic properties

- Holding all else constant, increasing surge multiplier should not reduce the pre-discount fare.
- For the same pricing zone and time, a longer distance should not reduce the distance component of fare.

- Applying a valid discount should not increase fare; cap and tax order must be specified before writing the property.
- Splitting or pooling a ride may change per-passenger price, so do not assert that pooled and solo totals are identical.
- For every output, fare must remain non-negative and final cap/tax behaviour must follow the clarified business rule.

Decision: MODIFY, with a model-based direction

Recommended decision: MODIFY the existing hybrid suite immediately by adding decision-table rules for new charges, boundary tests for surge values 1.0 and 3.0, promo time windows, GST and loyalty tiers, plus pairwise/orthogonal-array coverage. In parallel, develop a model-based generator and metamorphic checks for the growing rule space. This staged choice preserves regression protection while building toward redesign.

The choice must be defended with local measurements: test count, faults found, total runtime, coverage and maintenance time. Run the old suite unchanged against the new implementation and record which new-rule faults it misses. Recompute cyclomatic complexity from the new CFG; do not assume V(G)=35 merely because there are about 35 decision points, although predicate count + 1 can provide an initial estimate if each point is a simple predicate.

Metric	Old suite unchanged	Modified suite	Model-based candidate
New-rule faults detected	____ / ____	____ / ____	____ / ____
Number of tests	____	____	____
Full run time	____ min	____ min	____ min
Coverage / mutation score	____	____	____
Maintenance effort per release	____ min	____ min	____ min

The decision should change to REDESIGN if the modified suite cannot detect high-risk new-rule faults or cannot stay below 10 minutes despite prioritization and test parallelization. AI-assisted generation is useful for proposing combinations and properties, but generated tests must be reviewed for correct tax/discount ordering, pooled fare semantics, date/time-zone edges, and independently computed expected values.

12. Overall Recommendation

SwiftRide should use a hybrid strategy. Begin with specification-based black-box tests and robust boundary values, use decision tables for fare/promo/payment combinations and state-transition tests for lifecycle behaviour, then measure branch coverage and add white-box tests for uncovered logic. Mutation testing should challenge the strength of the suite, especially around the six reported defect patterns.

For the changed system, retain the existing critical regression cases, modify the suite with decision tables and pairwise/orthogonal arrays, and build toward model-based and metamorphic testing. Enforce the two-second per-test and ten-minute full-regression constraints with measured timing, not assumptions.

References and Tools

- Assignment 2 source specification: CSA37 – Software Testing, SwiftRide scenario and Tasks 1–10.
- Python 3 – implementation and test execution.
- pytest – automated test runner.
- coverage.py – statement and branch coverage.
- Hypothesis – property-based test generation.
- radon – cyclomatic complexity analysis.
- mutmut – mutation testing (optional).

Submission checklist: attach the actual test code, terminal output, coverage report, mutation report and screenshots or exported results from your own execution. Replace every blank in the measurement tables with real results and update any illustrative/predicted statement accordingly.

ANNEXURE A2.01 – Fare calculation: flat slab

Purpose. This worked example demonstrates how to design a focused test for fare calculation: flat slab in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

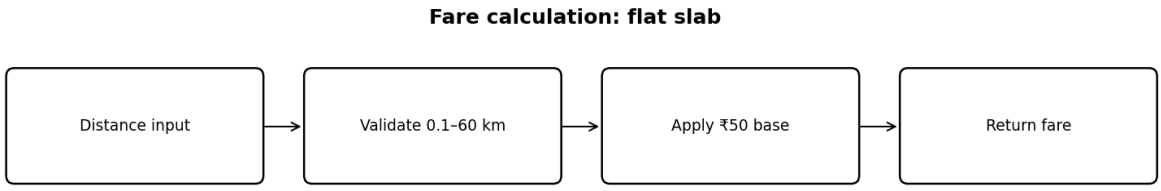


Figure A2.01. Fare calculation: flat slab – illustrative test flow.

Representative code checks

```
# Example 1: focused check
assert calculate_fare(1.0, 0, 12, '', 0, 'CASH') == 50.00

# Example 2: paired or boundary check
assert calculate_fare(2.1, 0, 12, '', 0, 'CASH') == 51.20
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.02 – Fare calculation: middle slab

Purpose. This worked example demonstrates how to design a focused test for fare calculation: middle slab in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

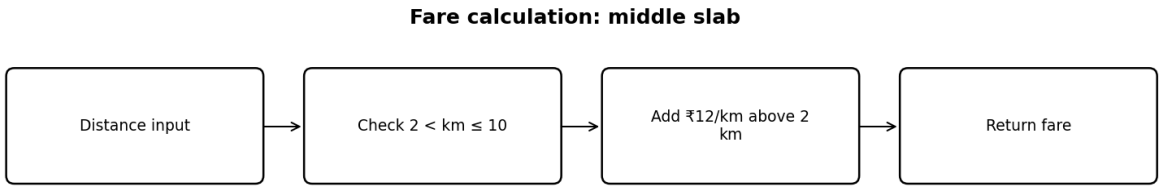


Figure A2.02. Fare calculation: middle slab – illustrative test flow.

Representative code checks

```
# Example 3: focused check
assert calculate_fare(5.0, 0, 12, '', 0, 'CASH') == 86.00

# Example 4: paired or boundary check
assert calculate_fare(10.0, 0, 12, '', 0, 'CASH') == 146.00
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.03 – Fare calculation: long slab

Purpose. This worked example demonstrates how to design a focused test for fare calculation: long slab in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

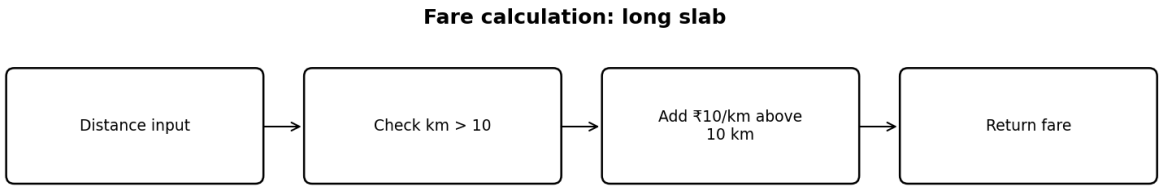


Figure A2.03. Fare calculation: long slab – illustrative test flow.

Representative code checks

```
# Example 5: focused check
assert calculate_fare(20.0, 0, 12, '', 0, 'CASH') == 246.00

# Example 6: paired or boundary check
assert calculate_fare(10.1, 0, 12, '', 0, 'CASH') == 147.00
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.04 – Waiting-time charge

Purpose. This worked example demonstrates how to design a focused test for waiting-time charge in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

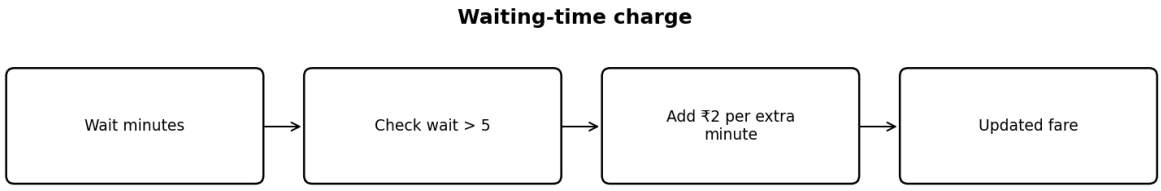


Figure A2.04. Waiting-time charge – illustrative test flow.

Representative code checks

```
# Example 7: focused check
assert calculate_fare(3, 5, 12, '', 0, 'CASH') == 62.00

# Example 8: paired or boundary check
assert calculate_fare(3, 6, 12, '', 0, 'CASH') == 64.00
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.05 – Night surcharge

Purpose. This worked example demonstrates how to design a focused test for night surcharge in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

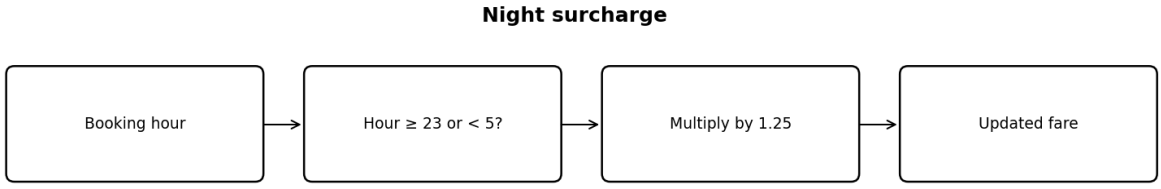


Figure A2.05. Night surcharge – illustrative test flow.

Representative code checks

```
# Example 9: focused check
assert calculate_fare(3, 0, 23, '', 0, 'CASH') == 77.50

# Example 10: paired or boundary check
assert calculate_fare(3, 0, 5, '', 0, 'CASH') == 62.00
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.06 – Promotion threshold

Purpose. This worked example demonstrates how to design a focused test for promotion threshold in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

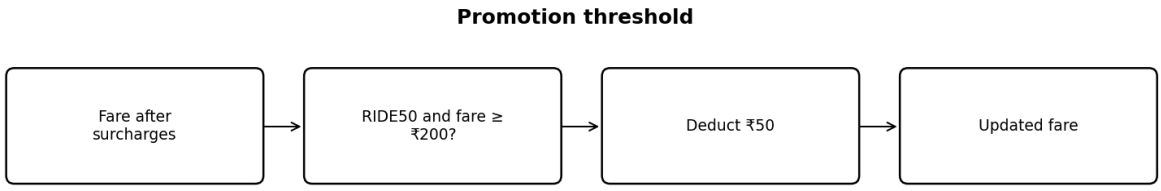


Figure A2.06. Promotion threshold – illustrative test flow.

Representative code checks

```
# Example 11: focused check
assert calculate_fare(20, 0, 12, 'RIDE50', 0, 'CASH') == 196.00

# Example 12: paired or boundary check
assert calculate_fare(20, 0, 12, '', 0, 'CASH') == 246.00
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.07 – Fare cap

Purpose. This worked example demonstrates how to design a focused test for fare cap in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

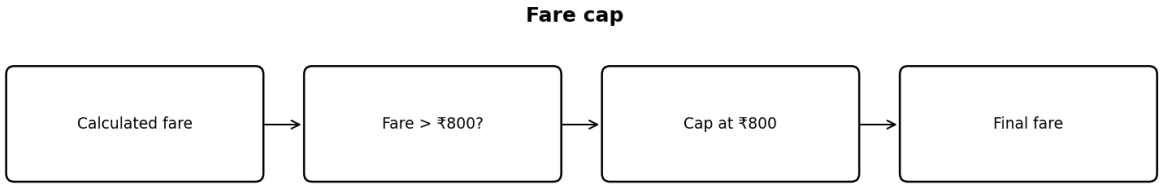


Figure A2.07. Fare cap – illustrative test flow.

Representative code checks

```
# Example 13: focused check
assert calculate_fare(60, 30, 23, '', 0, 'CASH') == 800.00

# Example 14: paired or boundary check
assert calculate_fare(60, 30, 12, '', 0, 'CASH') == 696.00
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.08 – Wallet validation

Purpose. This worked example demonstrates how to design a focused test for wallet validation in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

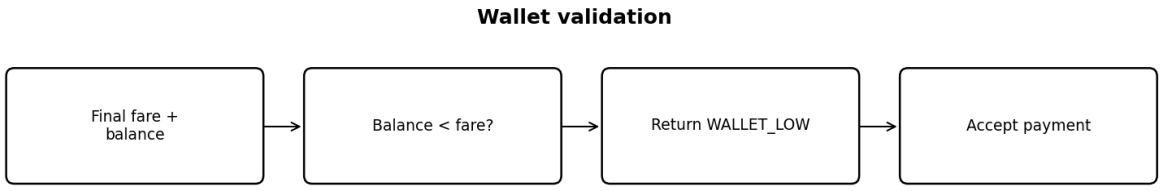


Figure A2.08. Wallet validation – illustrative test flow.

Representative code checks

```
# Example 15: focused check
assert calculate_fare(5, 0, 12, '', 88, 'WALLET') == 86.00

# Example 16: paired or boundary check
assert calculate_fare(5, 0, 12, '', 85, 'WALLET') == 'WALLET_LOW'
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.09 – Invalid distance

Purpose. This worked example demonstrates how to design a focused test for invalid distance in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

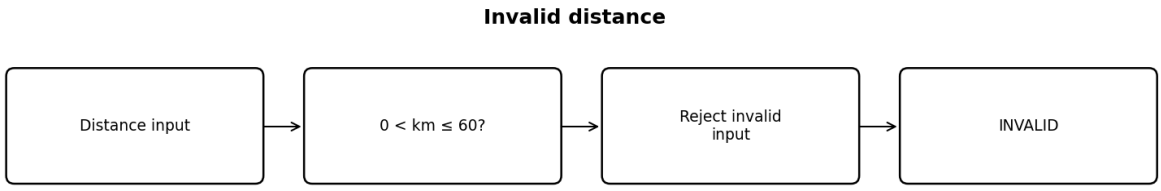


Figure A2.09. Invalid distance – illustrative test flow.

Representative code checks

```
# Example 17: focused check
assert calculate_fare(0, 0, 12, '', 0, 'CASH') == 'INVALID'

# Example 18: paired or boundary check
assert calculate_fare(60.1, 0, 12, '', 0, 'CASH') == 'INVALID'
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.10 – Exact slab boundary

Purpose. This worked example demonstrates how to design a focused test for exact slab boundary in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

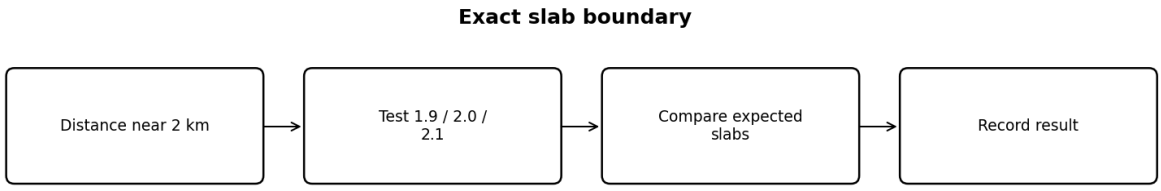


Figure A2.10. Exact slab boundary – illustrative test flow.

Representative code checks

```
# Example 19: focused check
for km in [1.9, 2.0, 2.1]:
    print(km, calculate_fare(km, 0, 12, '', 0, 'CASH'))

# Example 20: paired or boundary check
assert calculate_fare(2.0, 0, 12, '', 0, 'CASH') == 50.00
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.11 – Ten-kilometre boundary

Purpose. This worked example demonstrates how to design a focused test for ten-kilometre boundary in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

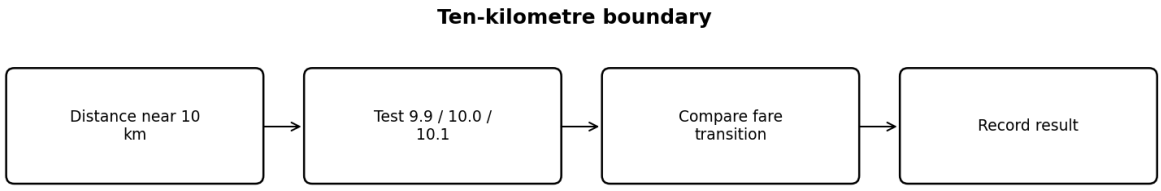


Figure A2.11. Ten-kilometre boundary – illustrative test flow.

Representative code checks

```
# Example 21: focused check
for km in [9.9, 10.0, 10.1]:
    print(km, calculate_fare(km, 0, 12, '', 0, 'CASH'))

# Example 22: paired or boundary check
assert calculate_fare(10.1, 0, 12, '', 0, 'CASH') == 147.00
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.12 – Night boundary

Purpose. This worked example demonstrates how to design a focused test for night boundary in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

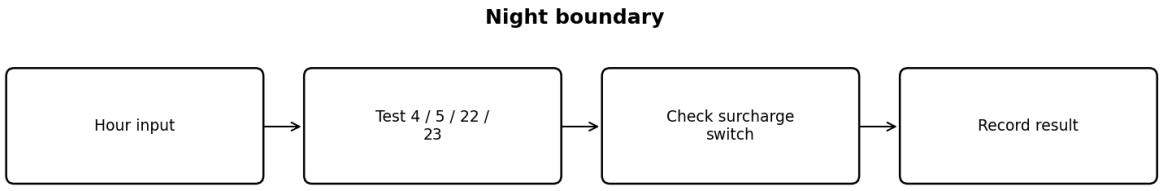


Figure A2.12. Night boundary – illustrative test flow.

Representative code checks

```
# Example 23: focused check
for hour in [4, 5, 22, 23]:
    print(hour, calculate_fare(3, 0, hour, '', 0, 'CASH'))

# Example 24: paired or boundary check
assert calculate_fare(3, 0, 23, '', 0, 'CASH') == 62.50
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.13 – Promo boundary

Purpose. This worked example demonstrates how to design a focused test for promo boundary in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

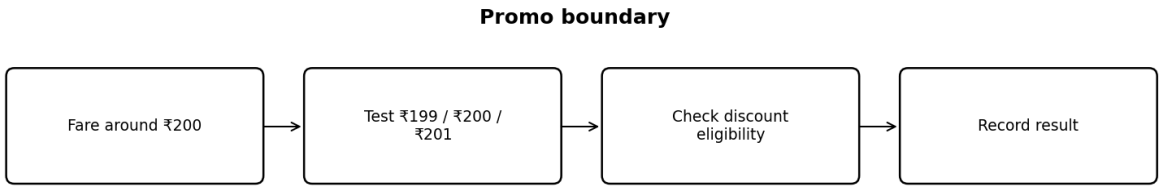


Figure A2.13. Promo boundary – illustrative test flow.

Representative code checks

```
# Example 25: focused check
assert calculate_fare(20, 0, 12, 'RIDE50', 0, 'CASH') == 196.00

# Example 26: paired or boundary check
assert calculate_fare(12, 0, 12, 'RIDE50', 0, 'CASH') == 166.00
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.14 – Wallet equality

Purpose. This worked example demonstrates how to design a focused test for wallet equality in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

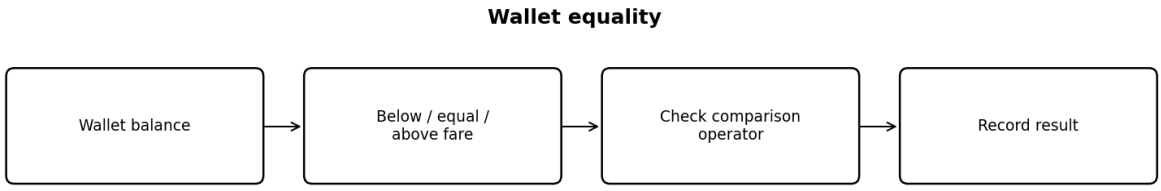


Figure A2.14. Wallet equality – illustrative test flow.

Representative code checks

```
# Example 27: focused check
assert calculate_fare(5, 0, 12, '', 86, 'WALLET') == 86.00

# Example 28: paired or boundary check
assert calculate_fare(5, 0, 12, '', 85.99, 'WALLET') == 'WALLET_LOW'
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.15 – Random test generation

Purpose. This worked example demonstrates how to design a focused test for random test generation in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

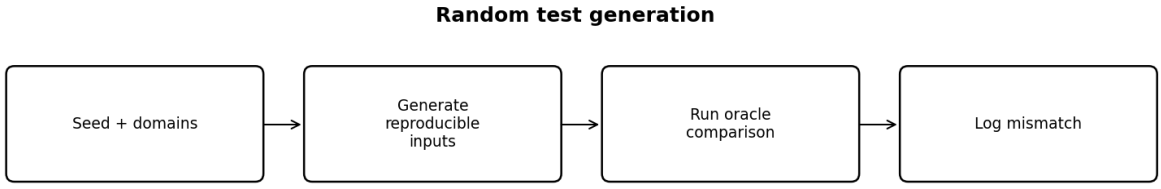


Figure A2.15. Random test generation – illustrative test flow.

Representative code checks

```
# Example 29: focused check
import random
rng = random.Random(42)
km = round(rng.uniform(0.1, 60), 1)

# Example 30: paired or boundary check
assert 0.1 <= km <= 60.0
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.16 – Equivalence partitions

Purpose. This worked example demonstrates how to design a focused test for equivalence partitions in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

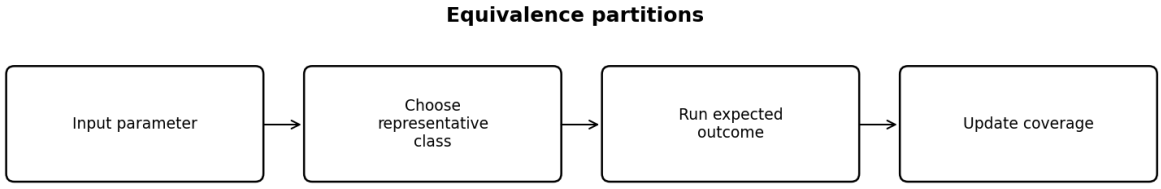


Figure A2.16. Equivalence partitions – illustrative test flow.

Representative code checks

```
# Example 31: focused check
classes = {'short': [0.1, 2.0], 'middle': [2.1, 10.0], 'long': [10.1, 60.0]}

# Example 32: paired or boundary check
assert classes['middle'][0] > 2.0
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.17 – Robust partitioning

Purpose. This worked example demonstrates how to design a focused test for robust partitioning in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

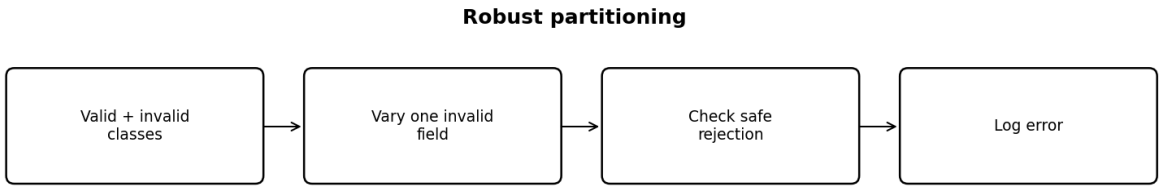


Figure A2.17. Robust partitioning – illustrative test flow.

Representative code checks

```
# Example 33: focused check
for bad_km in [0, -1, 60.1]:
    assert calculate_fare(bad_km, 0, 12, '', 0, 'CASH') == 'INVALID'

# Example 34: paired or boundary check
assert calculate_fare(5, 31, 12, '', 0, 'CASH') != 'INVALID'  # wait validation is separate
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.18 – Decision table

Purpose. This worked example demonstrates how to design a focused test for decision table in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

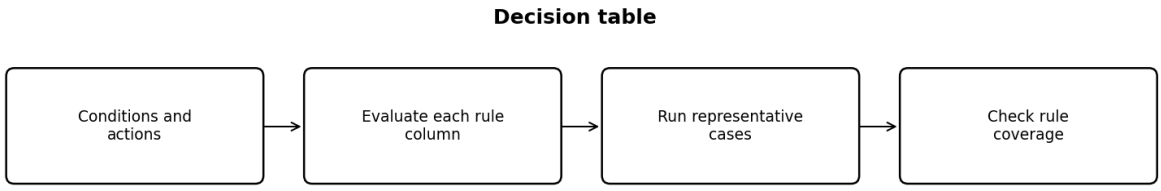


Figure A2.18. Decision table – illustrative test flow.

Representative code checks

```
# Example 35: focused check
rules = {'promo_match': True, 'fare_at_least_200': True}

# Example 36: paired or boundary check
assert all(k in rules for k in ['promo_match', 'fare_at_least_200'])
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.19 – Pairwise coverage

Purpose. This worked example demonstrates how to design a focused test for pairwise coverage in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

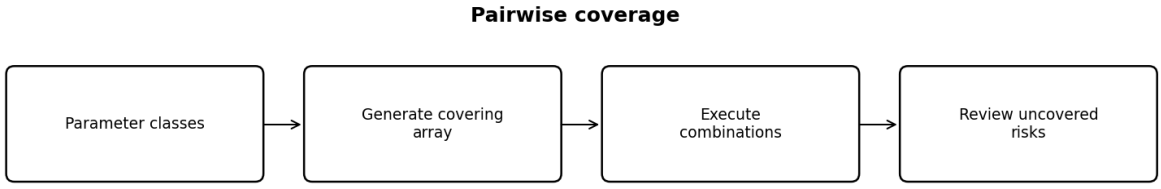


Figure A2.19. Pairwise coverage – illustrative test flow.

Representative code checks

```
# Example 37: focused check
dimensions = {'km':['short','middle','long'], 'wait':['free','charged'], 'hour':['day','night']}

# Example 38: paired or boundary check
assert len(dimensions) == 3
```

Test design and evidence

Check type	Test design	Expected evidence
Coverage target	Identify statements, branches or conditions before running.	Target and report use the same denominator.
Missing element	Add a test that reaches the uncovered outcome.	Coverage report explains the improvement.
Evidence	Save command, report and tool version.	Measured results can be independently checked.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.20 – State transition: booking

Purpose. This worked example demonstrates how to design a focused test for state transition: booking in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

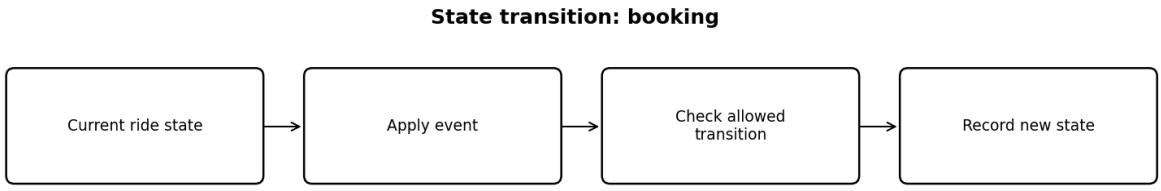


Figure A2.20. State transition: booking – illustrative test flow.

Representative code checks

```
# Example 39: focused check
allowed = {'Requested':['Accepted','Cancelled'], 'Accepted':['Arrived','Cancelled'],
'Arrived':['Started','Cancelled'], 'Started':['Completed'], 'Completed':[]}

# Example 40: paired or boundary check
assert 'Cancelled' in allowed['Arrived'] and 'Cancelled' not in allowed['Started']
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.21 – Cancellation policy

Purpose. This worked example demonstrates how to design a focused test for cancellation policy in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

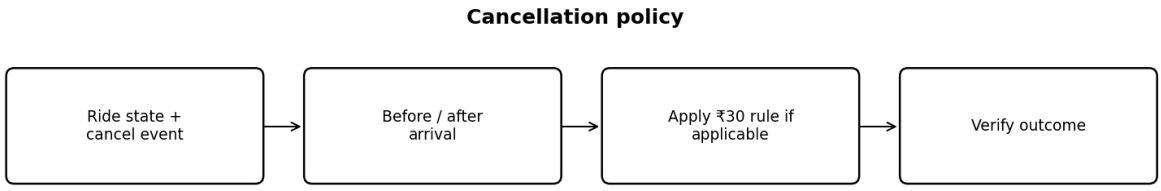


Figure A2.21. Cancellation policy – illustrative test flow.

Representative code checks

```
# Example 41: focused check
assert find_driver([2, 5, 8], 5) == 1

# Example 42: paired or boundary check
assert find_driver([], 5) == -1
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.22 – Binary search: found

Purpose. This worked example demonstrates how to design a focused test for binary search: found in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

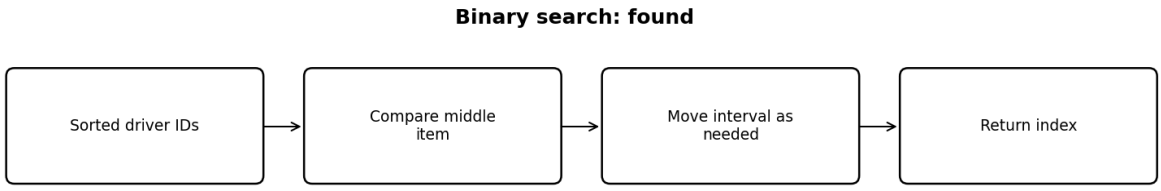


Figure A2.22. Binary search: found – illustrative test flow.

Representative code checks

```
# Example 43: focused check
assert find_driver([2, 5, 8], 8) == 2

# Example 44: paired or boundary check
assert find_driver([2, 5, 8], 4) == -1
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.23 – Binary search: not found

Purpose. This worked example demonstrates how to design a focused test for binary search: not found in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

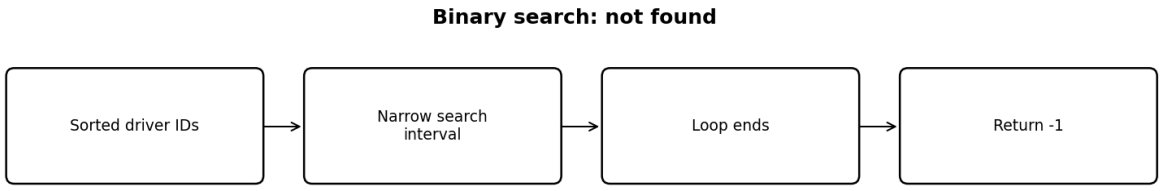


Figure A2.23. Binary search: not found – illustrative test flow.

Representative code checks

```
# Example 45: focused check
pytest.main(['-q'])

# Example 46: paired or boundary check
pytest.main(['--durations=10', '-q'])
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.24 – Statement coverage

Purpose. This worked example demonstrates how to design a focused test for statement coverage in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

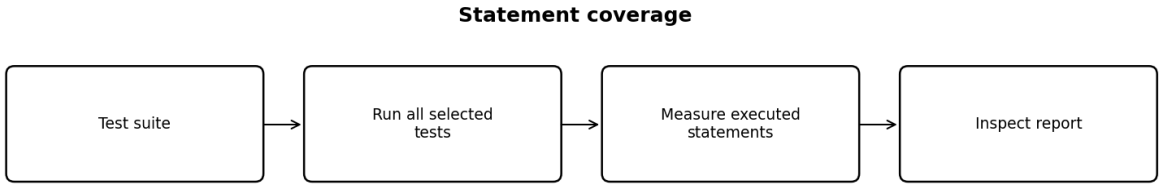


Figure A2.24. Statement coverage – illustrative test flow.

Representative code checks

```
# Example 47: focused check
pytest.main(['--cov=swiftride', '--cov-branch', '--cov-report=term-missing'])

# Example 48: paired or boundary check
pytest.main(['--cov=swiftride', '--cov-branch', '--cov-report=html'])
```

Test design and evidence

Check type	Test design	Expected evidence
Coverage target	Identify statements, branches or conditions before running.	Target and report use the same denominator.
Missing element	Add a test that reaches the uncovered outcome.	Coverage report explains the improvement.
Evidence	Save command, report and tool version.	Measured results can be independently checked.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.25 – Branch coverage

Purpose. This worked example demonstrates how to design a focused test for branch coverage in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

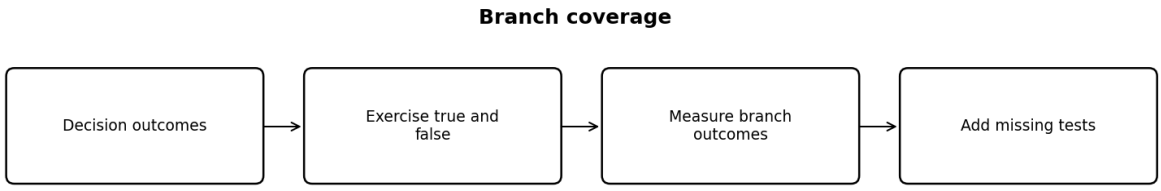


Figure A2.25. Branch coverage – illustrative test flow.

Representative code checks

```
# Example 49: focused check
# Example mutant: replace `km <= 2` with `km < 2`
# Boundary test km=2.0 should kill the mutant.

# Example 50: paired or boundary check
assert calculate_fare(2.0, 0, 12, '', 0, 'CASH') == 50.00
```

Test design and evidence

Check type	Test design	Expected evidence
Coverage target	Identify statements, branches or conditions before running.	Target and report use the same denominator.
Missing element	Add a test that reaches the uncovered outcome.	Coverage report explains the improvement.
Evidence	Save command, report and tool version.	Measured results can be independently checked.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.26 – Mutation testing

Purpose. This worked example demonstrates how to design a focused test for mutation testing in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

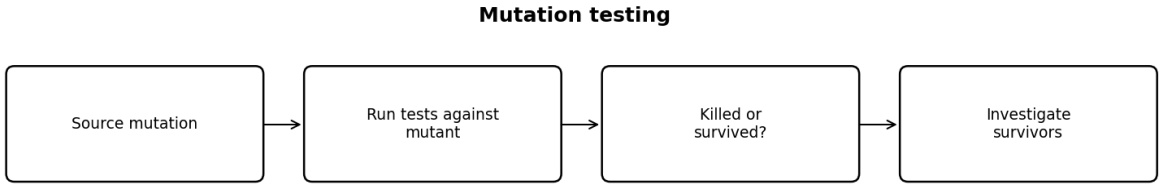


Figure A2.26. Mutation testing – illustrative test flow.

Representative code checks

```
# Example 51: focused check
assert (True or False) is True  # decision example only

# Example 52: paired or boundary check
assert (False or False) is False
```

Test design and evidence

Check type	Test design	Expected evidence
Coverage target	Identify statements, branches or conditions before running.	Target and report use the same denominator.
Missing element	Add a test that reaches the uncovered outcome.	Coverage report explains the improvement.
Evidence	Save command, report and tool version.	Measured results can be independently checked.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.27 – MC/DC condition A

Purpose. This worked example demonstrates how to design a focused test for mc/dc condition a in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

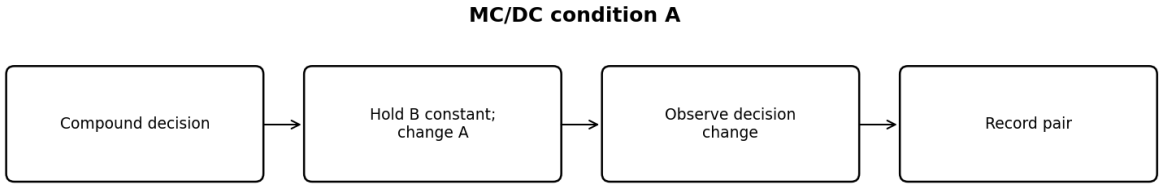


Figure A2.27. MC/DC condition A – illustrative test flow.

Representative code checks

```
# Example 53: focused check
assert (True and True) is True

# Example 54: paired or boundary check
assert (True and False) is False
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.28 – MC/DC condition B

Purpose. This worked example demonstrates how to design a focused test for mc/dc condition b in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

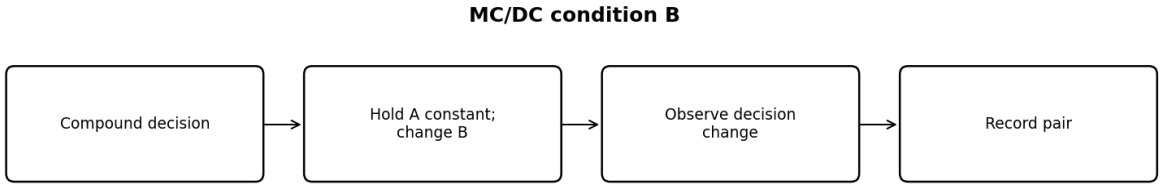


Figure A2.28. MC/DC condition B – illustrative test flow.

Representative code checks

```
# Example 55: focused check
# Nodes N, edges E, connected components P=1
V_G = E - N + 2

# Example 56: paired or boundary check
assert V_G == E - N + 2
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.29 – Control-flow graph

Purpose. This worked example demonstrates how to design a focused test for control-flow graph in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

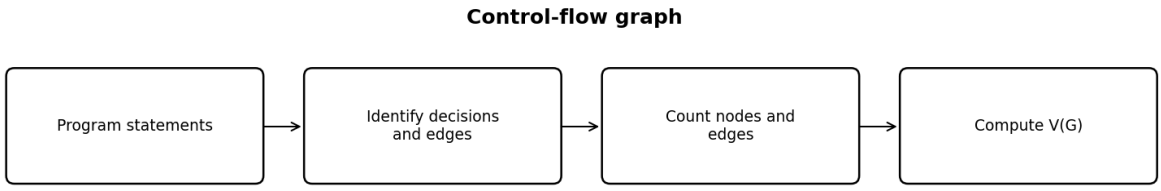


Figure A2.29. Control-flow graph – illustrative test flow.

Representative code checks

```
# Example 57: focused check
paths = ['empty-list', 'middle-hit', 'left-branch', 'right-branch']

# Example 58: paired or boundary check
assert len(set(paths)) == 4
```

Test design and evidence

Check type	Test design	Expected evidence
Coverage target	Identify statements, branches or conditions before running.	Target and report use the same denominator.
Missing element	Add a test that reaches the uncovered outcome.	Coverage report explains the improvement.
Evidence	Save command, report and tool version.	Measured results can be independently checked.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.30 – Basis path testing

Purpose. This worked example demonstrates how to design a focused test for basis path testing in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

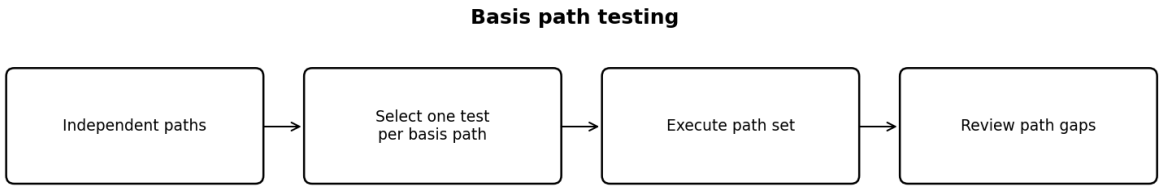


Figure A2.30. Basis path testing – illustrative test flow.

Representative code checks

```
# Example 59: focused check
critical = ['fare-boundaries', 'night-surcharge', 'promo', 'wallet']

# Example 60: paired or boundary check
assert critical[0] == 'fare-boundaries'
```

Test design and evidence

Check type	Test design	Expected evidence
Coverage target	Identify statements, branches or conditions before running.	Target and report use the same denominator.
Missing element	Add a test that reaches the uncovered outcome.	Coverage report explains the improvement.
Evidence	Save command, report and tool version.	Measured results can be independently checked.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.31 – Regression prioritization

Purpose. This worked example demonstrates how to design a focused test for regression prioritization in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

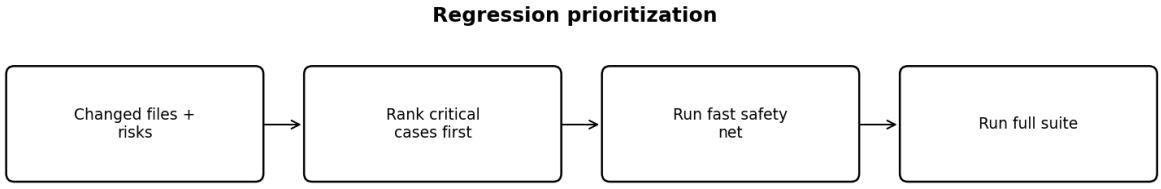


Figure A2.31. Regression prioritization – illustrative test flow.

Representative code checks

```
# Example 61: focused check
steps = ['checkout', 'install', 'test', 'coverage', 'publish']

# Example 62: paired or boundary check
assert steps[-1] == 'publish'
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.32 – CI pipeline

Purpose. This worked example demonstrates how to design a focused test for ci pipeline in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

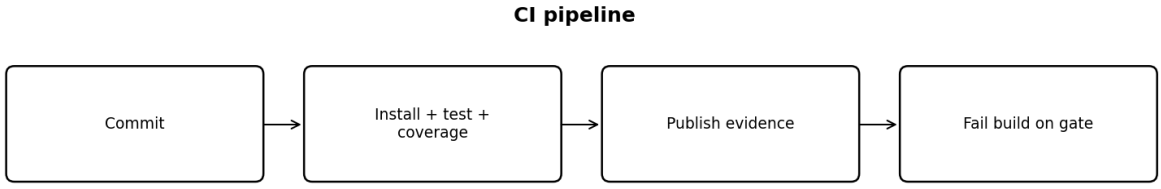


Figure A2.32. CI pipeline – illustrative test flow.

Representative code checks

```
# Example 63: focused check
def classify_defect(reproducible, impact):
    return 'confirmed' if reproducible and impact else 'triage'

# Example 64: paired or boundary check
assert classify_defect(True, True) == 'confirmed'
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.33 – Defect lifecycle

Purpose. This worked example demonstrates how to design a focused test for defect lifecycle in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

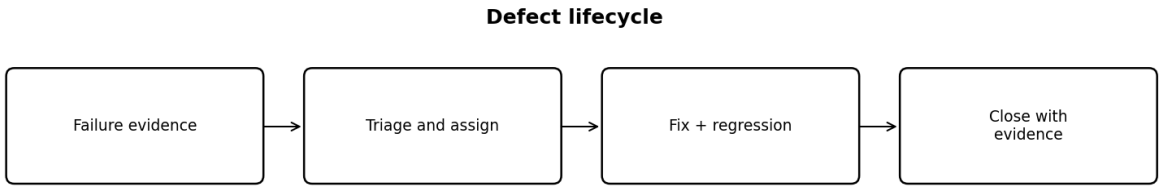


Figure A2.33. Defect lifecycle – illustrative test flow.

Representative code checks

```
# Example 65: focused check
evidence = {'test_id':'SR-001', 'build':'local', 'status':'Not Run'}

# Example 66: paired or boundary check
assert evidence['status'] in ['Pass','Fail','Blocked','Not Run']
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.34 – Test evidence

Purpose. This worked example demonstrates how to design a focused test for test evidence in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

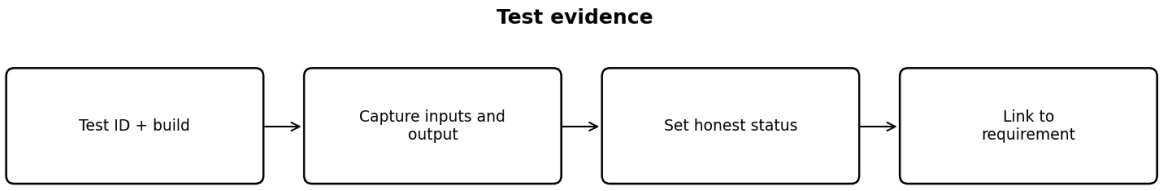


Figure A2.34. Test evidence – illustrative test flow.

Representative code checks

```
# Example 67: focused check
mandatory_gates = {'critical_tests': True, 'open_blocker': False}

# Example 68: paired or boundary check
assert mandatory_gates['critical_tests'] and not mandatory_gates['open_blocker']
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.35 – Release readiness

Purpose. This worked example demonstrates how to design a focused test for release readiness in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

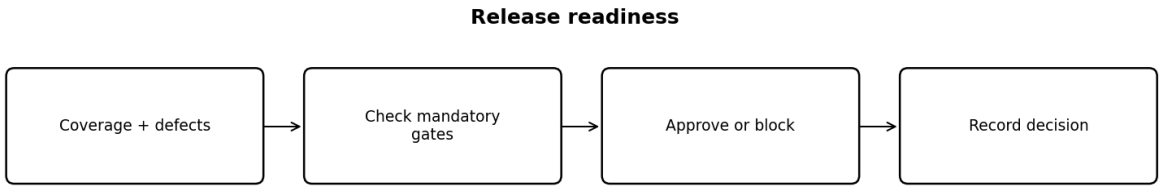


Figure A2.35. Release readiness – illustrative test flow.

Representative code checks

```
# Example 69: focused check
assert not (wait < 0 or wait > 30)  # valid-domain assertion

# Example 70: paired or boundary check
assert 0 <= 30 <= 30
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.36 – Input validation for wait time

Purpose. This worked example demonstrates how to design a focused test for input validation for wait time in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

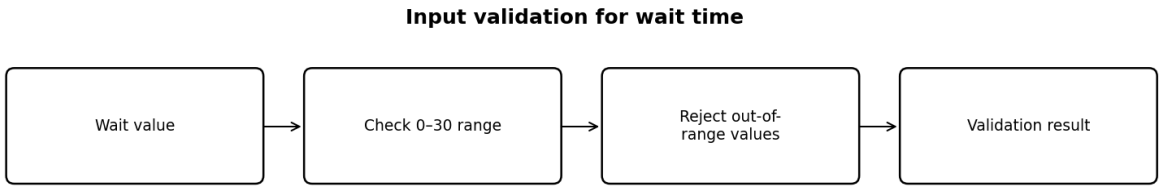


Figure A2.36. Input validation for wait time – illustrative test flow.

Representative code checks

```
# Example 71: focused check
assert 0 <= 23 <= 23

# Example 72: paired or boundary check
assert not (24 <= 23)
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.37 – Input validation for hour

Purpose. This worked example demonstrates how to design a focused test for input validation for hour in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

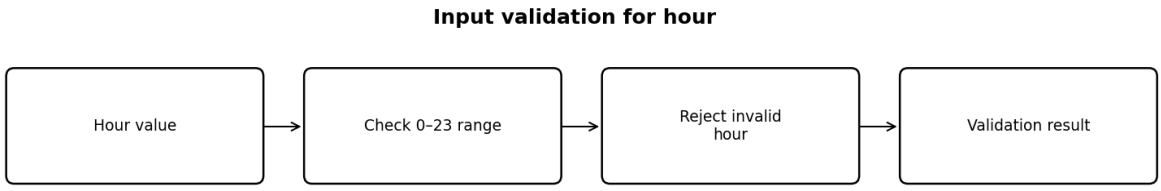


Figure A2.37. Input validation for hour – illustrative test flow.

Representative code checks

```
# Example 73: focused check
assert mode in ['WALLET', 'UPI', 'CASH']

# Example 74: paired or boundary check
assert 'CRYPTO' not in ['WALLET', 'UPI', 'CASH']
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.38 – Payment-mode partitioning

Purpose. This worked example demonstrates how to design a focused test for payment-mode partitioning in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

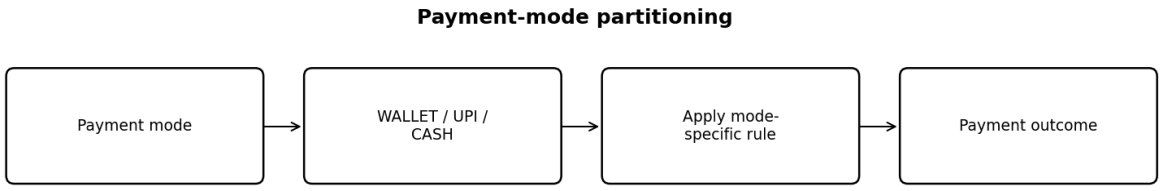


Figure A2.38. Payment-mode partitioning – illustrative test flow.

Representative code checks

```
# Example 75: focused check
used_codes = set()
assert 'RIDE50' not in used_codes

# Example 76: paired or boundary check
used_codes.add('RIDE50')
assert 'RIDE50' in used_codes
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.39 – Promo single-use rule

Purpose. This worked example demonstrates how to design a focused test for promo single-use rule in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

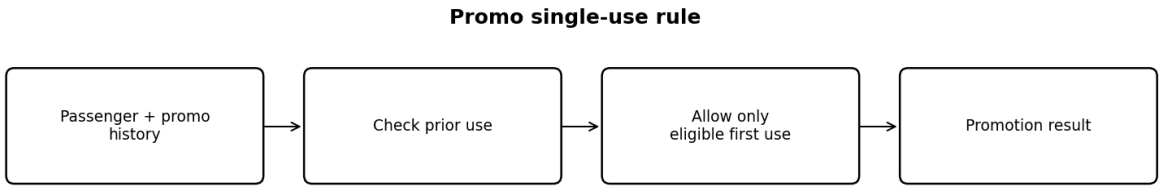


Figure A2.39. Promo single-use rule – illustrative test flow.

Representative code checks

```
# Example 77: focused check
fare = round(51.2, 2)
assert fare == 51.20

# Example 78: paired or boundary check
assert round(147.0, 2) == 147.00
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.40 – Rounding policy

Purpose. This worked example demonstrates how to design a focused test for rounding policy in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

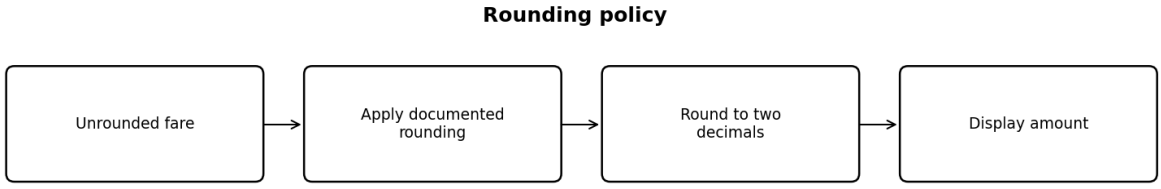


Figure A2.40. Rounding policy – illustrative test flow.

Representative code checks

```
# Example 79: focused check
# Keep calculation order aligned to the approved requirement
# document the order before encoding assertions.

# Example 80: paired or boundary check
assert 'discount order' in 'document discount order'
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.41 – Tax/discount ordering change

Purpose. This worked example demonstrates how to design a focused test for tax/discount ordering change in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

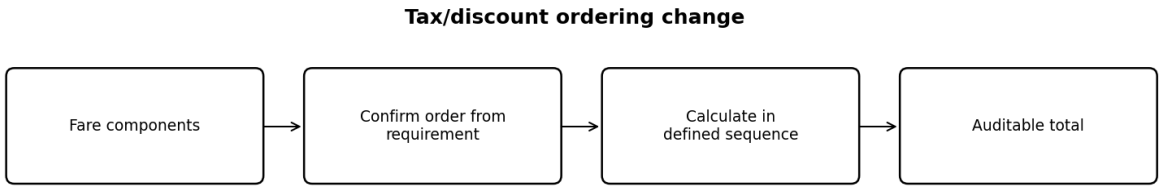


Figure A2.41. Tax/discount ordering change – illustrative test flow.

Representative code checks

```
# Example 81: focused check
# Do not assume pooled and solo fares are equal.
# Compare against the approved allocation policy.

# Example 82: paired or boundary check
assert 0 <= 2.0 <= 3.0  # multiplier-domain example
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.42 – Pooled ride risk

Purpose. This worked example demonstrates how to design a focused test for pooled ride risk in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

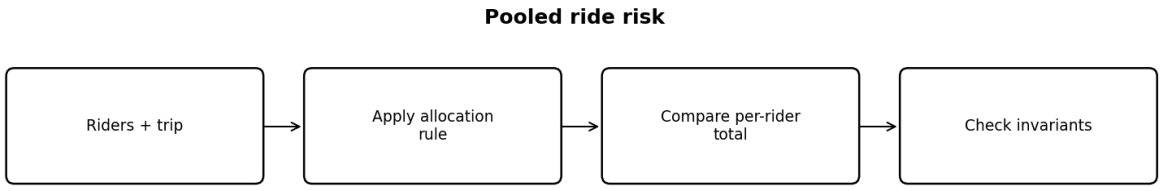


Figure A2.42. Pooled ride risk – illustrative test flow.

Representative code checks

```
# Example 83: focused check
for surge in [1.0, 1.1, 2.9, 3.0]:
    assert 1.0 <= surge <= 3.0

# Example 84: paired or boundary check
assert 3.0 >= 1.0
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.43 – Surge multiplier boundary

Purpose. This worked example demonstrates how to design a focused test for surge multiplier boundary in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

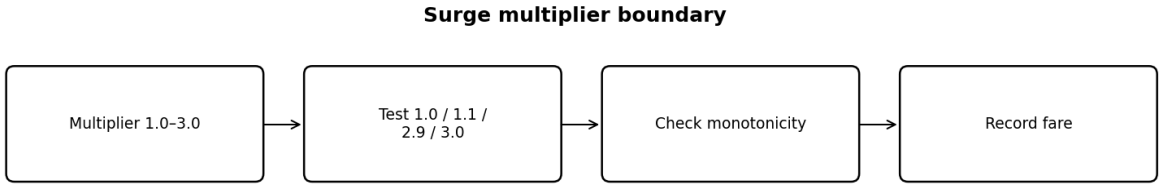


Figure A2.43. Surge multiplier boundary – illustrative test flow.

Representative code checks

```
# Example 85: focused check
assert promo_start <= booking_time <= promo_end

# Example 86: paired or boundary check
assert not (booking_time < promo_start)
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.44 – Time-window promotion

Purpose. This worked example demonstrates how to design a focused test for time-window promotion in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

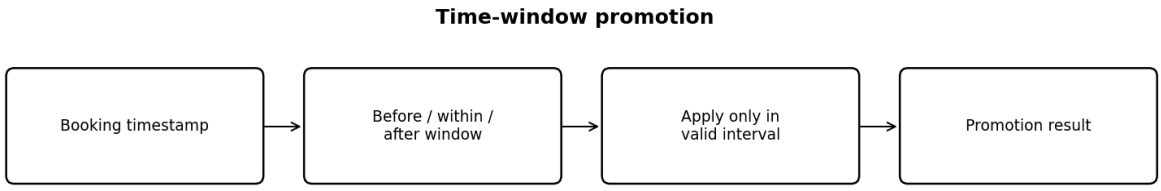


Figure A2.44. Time-window promotion – illustrative test flow.

Representative code checks

```
# Example 87: focused check
assert fare >= 0

# Example 88: paired or boundary check
assert fare <= 800
```

Test design and evidence

Check type	Test design	Expected evidence
Nominal / in-range	Use a representative valid value.	Expected fare follows the stated formula.
Exact boundary	Test the boundary and nearest permitted values on each side.	No off-by-one error; rule switches at the documented point.
Evidence	Save inputs, expected value, observed value and test status.	A reviewer can reproduce the result.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.45 – Property-based invariant

Purpose. This worked example demonstrates how to design a focused test for property-based invariant in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

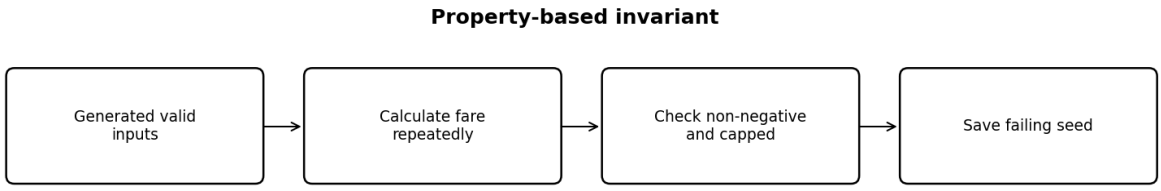


Figure A2.45. Property-based invariant – illustrative test flow.

Representative code checks

```
# Example 89: focused check
fixture = {'km': 2.0, 'wait': 0, 'hour': 12}

# Example 90: paired or boundary check
assert fixture['km'] == 2.0
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.46 – Test fixture design

Purpose. This worked example demonstrates how to design a focused test for test fixture design in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

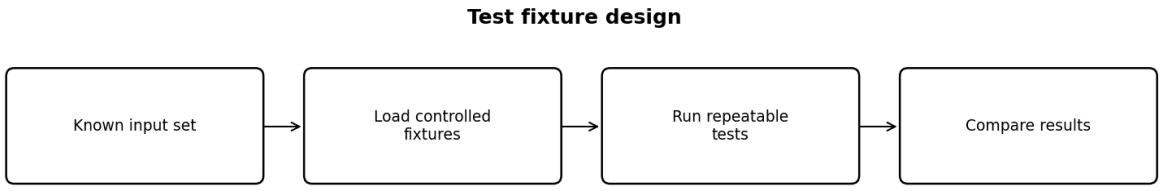


Figure A2.46. Test fixture design – illustrative test flow.

Representative code checks

```
# Example 91: focused check
def classify_failure(reproducible):
    return 'confirmed' if reproducible else 'needs-triage'

# Example 92: paired or boundary check
assert classify_failure(True) == 'confirmed'
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.47 – Failure triage

Purpose. This worked example demonstrates how to design a focused test for failure triage in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

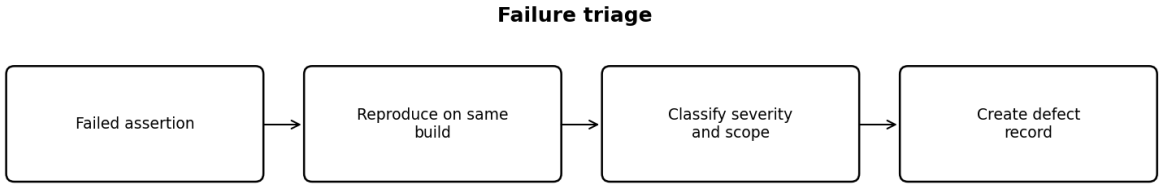


Figure A2.47. Failure triage – illustrative test flow.

Representative code checks

```
# Example 93: focused check
# Reset persistent state before each independent test
reset_test_database()

# Example 94: paired or boundary check
assert test_run_id != ''
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.48 – Test isolation

Purpose. This worked example demonstrates how to design a focused test for test isolation in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

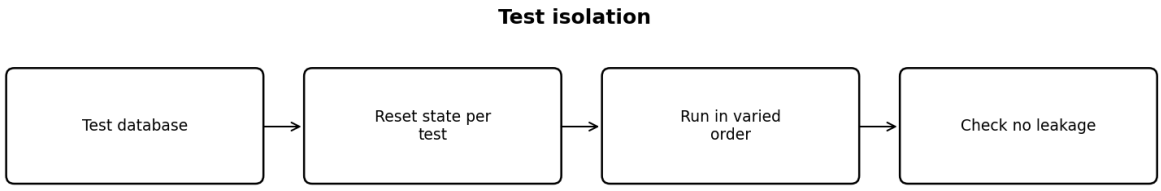


Figure A2.48. Test isolation – illustrative test flow.

Representative code checks

```
# Example 95: focused check
# Send two requests with the same slot concurrently
# Assert only one booking remains active.

# Example 96: paired or boundary check
assert active_bookings_for_slot <= 1
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. After the test is added, run it both alone and as part of the full suite. A case that passes only in a particular order may depend on shared state or leaked test data and should be isolated before release. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.49 – Concurrency risk

Purpose. This worked example demonstrates how to design a focused test for concurrency risk in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

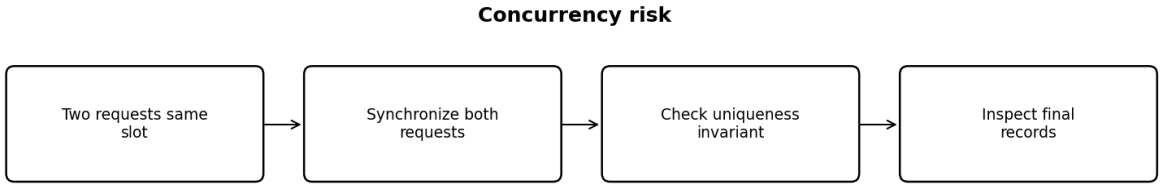


Figure A2.49. Concurrency risk – illustrative test flow.

Representative code checks

```
# Example 97: focused check
gate = {'critical': True, 'runtime_min': 8.5}

# Example 98: paired or boundary check
assert gate['critical'] and gate['runtime_min'] <= 10
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Interpret the expected result from the written rule first. The code sample is a template; connect it to the actual function and use an independent expected-value calculation. Keep each test independent so that a failure points to one specific rule. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.50 – Weekly regression gate

Purpose. This worked example demonstrates how to design a focused test for weekly regression gate in the SwiftRide scenario. It complements the relevant task in the main report by connecting a business rule or structural property to test inputs, an expected result, and evidence that can be reviewed later.

Process flow. The diagram shows the simplified sequence from input selection through rule evaluation to result recording. It is a design aid, not proof that the production application has been executed.

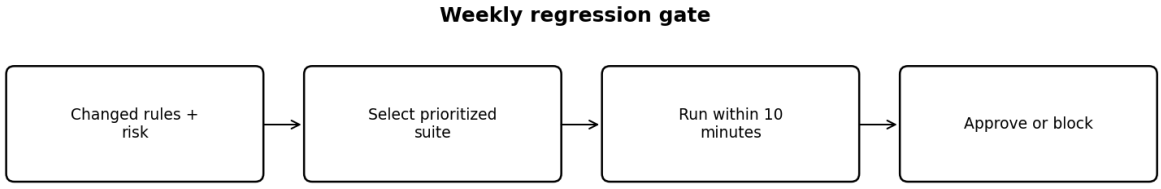


Figure A2.50. Weekly regression gate – illustrative test flow.

Representative code checks

```
# Example 99: focused check
assert fare >= 0  # replace with computed final fare from test case

# Example 100: paired or boundary check
assert fare <= 800  # cap applies after the specified fare adjustments
```

Test design and evidence

Check type	Test design	Expected evidence
Positive scenario	Choose a valid, representative input or event.	Expected behaviour occurs.
Negative / edge scenario	Use an invalid value, rejected event or important edge case.	System rejects safely or follows the specified alternative.
Evidence	Record input, result, environment and build.	Outcome is traceable and repeatable.

Interpretation and limitation. Record the input, expected output, actual output, test status, build identifier and execution timestamp. If the software or environment is unavailable, use Blocked or Not Run rather than marking the case as passed. For production use, confirm all business assumptions with the assignment specification or system owner; do not infer unprovided behaviour from the sample code.

Review checklist: requirement linked; expected result independently checked; input data controlled; negative case considered; execution status reported honestly; regression impact reviewed.

ANNEXURE A2.51 – API contract validation

Purpose. This worked example adds a practical validation view for api contract validation. It links the risk, a representative check, the expected outcome and the evidence required for review. The example is a test-design template and must be connected to the actual application interfaces and approved requirements before execution.

Process flow. Use the flow to follow the test from controlled input through evaluation to evidence capture. The diagram represents the intended verification sequence, not a claim that the live system was tested.

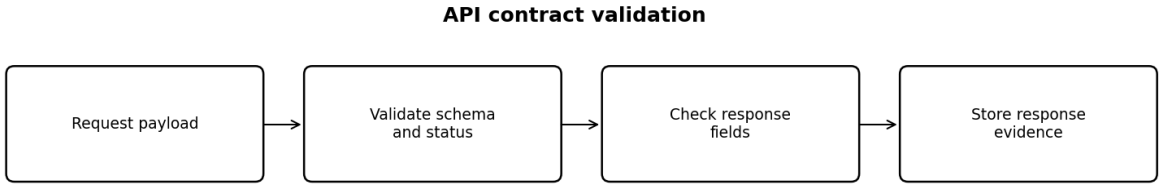


Figure A2.51. API contract validation – illustrative test flow.

Representative code checks

Test design and evidence

```
assert response.status_code in [200, 400, 401, 403, 422]

assert set(['status', 'fare']).issubset(response.json().keys())
```

Check	Test design	Evidence to retain
Positive case	Run an allowed, correctly formed request or action.	Input, response and expected result.
Negative / repeat case	Try a forbidden action, duplicate request or threshold case.	Rejection result, logs and record state.
Review gate	Compare result with the stated requirement.	Build ID, timestamp, environment and status.

Interpretation and limitation. The test should fail clearly when the required behaviour is violated. Avoid using a mock-only result as proof of end-to-end correctness where database, gateway, permissions or timing behaviour is part of the requirement. Use synthetic accounts and approved test environments.

Evidence checklist: requirement linked; input controlled; expected result reviewed; observed result captured; status recorded honestly; residual risk documented.

ANNEXURE A2.52 – Idempotent payment retry

Purpose. This worked example adds a practical validation view for idempotent payment retry. It links the risk, a representative check, the expected outcome and the evidence required for review. The example is a test-design template and must be connected to the actual application interfaces and approved requirements before execution.

Process flow. Use the flow to follow the test from controlled input through evaluation to evidence capture. The diagram represents the intended verification sequence, not a claim that the live system was tested.

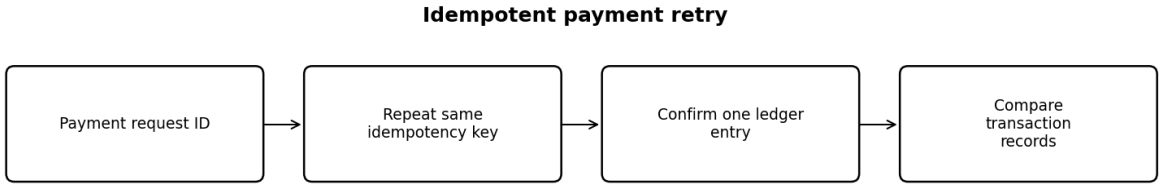


Figure A2.52. Idempotent payment retry – illustrative test flow.

Representative code checks

Test design and evidence

```
assert first_payment.idempotency_key == retry.idempotency_key

assert ledger.count_by_key(key) == 1
```

Check	Test design	Evidence to retain
Positive case	Run an allowed, correctly formed request or action.	Input, response and expected result.
Negative / repeat case	Try a forbidden action, duplicate request or threshold case.	Rejection result, logs and record state.
Review gate	Compare result with the stated requirement.	Build ID, timestamp, environment and status.

Interpretation and limitation. The test should fail clearly when the required behaviour is violated. Avoid using a mock-only result as proof of end-to-end correctness where database, gateway, permissions or timing behaviour is part of the requirement. Use synthetic accounts and approved test environments.

Evidence checklist: requirement linked; input controlled; expected result reviewed; observed result captured; status recorded honestly; residual risk documented.

ANNEXURE A2.53 – Security authorization

Purpose. This worked example adds a practical validation view for security authorization. It links the risk, a representative check, the expected outcome and the evidence required for review. The example is a test-design template and must be connected to the actual application interfaces and approved requirements before execution.

Process flow. Use the flow to follow the test from controlled input through evaluation to evidence capture. The diagram represents the intended verification sequence, not a claim that the live system was tested.

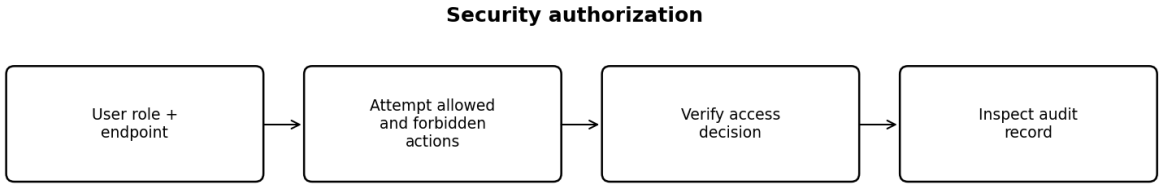


Figure A2.53. Security authorization – illustrative test flow.

Representative code checks

Test design and evidence

```
assert response_for_allowed_role.status_code == 200

assert response_for_forbidden_role.status_code in [401, 403]
```

Check	Test design	Evidence to retain
Positive case	Run an allowed, correctly formed request or action.	Input, response and expected result.
Negative / repeat case	Try a forbidden action, duplicate request or threshold case.	Rejection result, logs and record state.
Review gate	Compare result with the stated requirement.	Build ID, timestamp, environment and status.

Interpretation and limitation. The test should fail clearly when the required behaviour is violated. Avoid using a mock-only result as proof of end-to-end correctness where database, gateway, permissions or timing behaviour is part of the requirement. Use synthetic accounts and approved test environments.

Evidence checklist: requirement linked; input controlled; expected result reviewed; observed result captured; status recorded honestly; residual risk documented.

ANNEXURE A2.54 – Performance threshold

Purpose. This worked example adds a practical validation view for performance threshold. It links the risk, a representative check, the expected outcome and the evidence required for review. The example is a test-design template and must be connected to the actual application interfaces and approved requirements before execution.

Process flow. Use the flow to follow the test from controlled input through evaluation to evidence capture. The diagram represents the intended verification sequence, not a claim that the live system was tested.

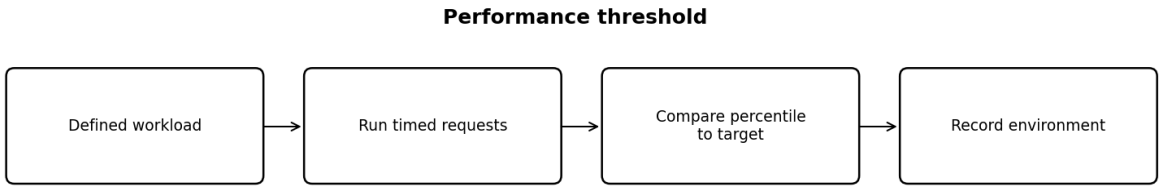


Figure A2.54. Performance threshold – illustrative test flow.

Representative code checks

Test design and evidence

```
assert elapsed_seconds < defined_threshold

assert percentile_95_ms <= agreed_p95_ms
```

Check	Test design	Evidence to retain
Positive case	Run an allowed, correctly formed request or action.	Input, response and expected result.
Negative / repeat case	Try a forbidden action, duplicate request or threshold case.	Rejection result, logs and record state.
Review gate	Compare result with the stated requirement.	Build ID, timestamp, environment and status.

Interpretation and limitation. The test should fail clearly when the required behaviour is violated. Avoid using a mock-only result as proof of end-to-end correctness where database, gateway, permissions or timing behaviour is part of the requirement. Use synthetic accounts and approved test environments.

Evidence checklist: requirement linked; input controlled; expected result reviewed; observed result captured; status recorded honestly; residual risk documented.

ANNEXURE A2.55 – Final submission audit

Purpose. This worked example adds a practical validation view for final submission audit. It links the risk, a representative check, the expected outcome and the evidence required for review. The example is a test-design template and must be connected to the actual application interfaces and approved requirements before execution.

Process flow. Use the flow to follow the test from controlled input through evaluation to evidence capture. The diagram represents the intended verification sequence, not a claim that the live system was tested.

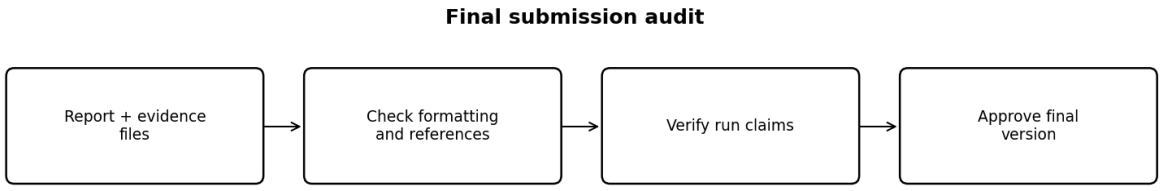


Figure A2.55. Final submission audit – illustrative test flow.

Representative code checks

Test design and evidence

```
assert all(item in checklist for item in required_items)

assert execution_claims_have_evidence is True
```

Check	Test design	Evidence to retain
Positive case	Run an allowed, correctly formed request or action.	Input, response and expected result.
Negative / repeat case	Try a forbidden action, duplicate request or threshold case.	Rejection result, logs and record state.
Review gate	Compare result with the stated requirement.	Build ID, timestamp, environment and status.

Interpretation and limitation. The test should fail clearly when the required behaviour is violated. Avoid using a mock-only result as proof of end-to-end correctness where database, gateway, permissions or timing behaviour is part of the requirement. Use synthetic accounts and approved test environments.

Evidence checklist: requirement linked; input controlled; expected result reviewed; observed result captured; status recorded honestly; residual risk documented.